

Appendix

B

Study Tips





This appendix covers suggestions and recommendations for how you should prepare for the certification exam. If you're an experienced test taker, or you've taken a certification test before, most of this should be common knowledge. For those who are taking the exam for the first time, don't panic! We'll present a number of tips and strategies in this appendix to help you prepare for the exam.

Studying for the Test

Before you even sign up and take the test, you need to study the material. Studying includes the following tasks:

- Create a study plan.
- Read the Study Guide material.
- Create and run sample applications.
- Solve the Review Questions at the end of each chapter.
- Create flashcards and/or use the ones we've provided.
- Take the three practice exams.

The book is divided into chapters with corresponding exam objectives, to make it easier to assimilate. The earlier chapters on syntax and operators are especially important since they are used throughout the code samples on the exam. Unless we explicitly stated something was out of scope for the exam, you will be required to have a strong understanding of all the information in this book.

Creating a Study Plan

Rome wasn't built in a day, so you shouldn't attempt to study for only one day. Even if you have been certified with a previous version of Java, the new test includes features and components unique to Java 8 that are covered in this text.

Once you have decided to take the test, which we assume you have already since you're reading this book, you should construct a study plan that fits with your schedule. We recommend you set aside some amount of time each day, even if it's just a few minutes during lunch, to read or practice for the exam. The idea is to keep your momentum going throughout the exam preparation process. The more consistent you are in how you study, the better prepared you will be for the exam. Try to avoid taking a few days or weeks off from studying, or you're likely to spend a lot of time relearning existing material instead of moving on to new material.

Let's say you begin studying on January 1. Assuming you allot two weeks per chapter, we constructed a study plan in Table B.1 that you can use as a schedule throughout the study process. Of course, if you're new to Java, two weeks per chapter may not be enough; if you're an experienced Java developer, you may only need a few days per chapter.

TABLE B.1 Sample study plan

Date	Task
January 1–January 11	Read Introduction, Appendix B, and Chapter 1
January 12–January 14	Answer Chapter 1 Review Questions
January 15–January 25	Read Chapter 2
January 26–January 28	Answer Chapter 2 Review Questions
January 29–February 8	Read Chapter 3
February 9–February 11	Answer Chapter 3 Review Questions
February 12–February 22	Read Chapter 4
February 23–February 25	Answer Chapter 4 Review Questions
February 26–March 8	Read Chapter 5
March 9–March 11	Answer Chapter 5 Review Questions
March 12–March 22	Read Chapter 6
March 23–March 25	Answer Chapter 6 Review Questions
March 26–April 2	Take practice exams and practice with flashcards
April 3	Take exam

Your own study plan will vary based on your familiarity with Java, your personal and work schedule, and your learning abilities. The idea is to create a plan early on that has self-imposed deadlines that you can follow throughout the studying process. When someone asks how you're doing preparing for the exam, you should have a strong sense of what you've learned so far, what you're currently studying, and how many weeks you need to be prepared to take the exam.

Creating and Running Sample Applications

Although some people can learn Java just by reading a textbook, that's not how we recommend you study for a certification exam. We want you to be writing your own Java sample

applications throughout this book so that you don't just learn the material but you also understand the material. For example, it may not be obvious why the following line of code does not compile, but if you try to compile it yourself, the Java compiler will tell you the problem.

```
float value = 102.0;    // DOES NOT COMPILE
```

In this section, we will discuss how to test Java code and the tools available to assist you in this process.



A lot of people post on the CodeRanch.com forum asking, “Why does this code not compile?” and we encourage you to post the compiler error message anytime you need help. We recommend you also read the compiler message when posting, since it may provide meaningful information about why the code failed to compile.

In the previous example, the compiler failed to compile with the message `Type mismatch: cannot convert from double to float`. This message indicates that we are trying to convert a double value, `102.0`, to a float variable reference using an implicit cast. If we add an explicit cast to `(float)` or change the value to `102.0f`, the code will compile without issue.

Sample Test Class

Throughout this book, we present numerous code snippets and ask you whether they'll compile and what their output will be. These snippets are designed to be placed inside a simple Java application that starts, executes the code, and terminates. As described in Chapter 1, “Java Building Blocks,” you can accomplish this by compiling and running a public class containing a public static void `main(String[] args)` method, such as the following:

```
public class TestClass {
    public static void main(String[] args) {
        // Add test code here

        // Add any print statements here
        System.out.println("Hello World!");
    }
}
```

This application isn't particularly interesting—it just outputs “Hello World” and exits. That said, we can insert many of the code snippets present in this book in the `main()` method to determine if the code compiles, as well as what the code outputs when it does compile. We strongly recommend you become familiar with this sample application, so much so that you could write it from memory, without the comments.

We recommend that while reading this book you make note of any sections that you do not fully understand and revisit them when in front of a computer screen with a Java compiler and Java runtime. You should start by copying the code snippet into your test class, and then try experimenting with the code as much as possible. For example, we indicated the previous sample line of code would not compile, but would any of the following compile?

```
float value1 = 102;  
float value2 = (int)102.0;  
float value3 = 1f * 0.0;  
float value4 = 1f * (short)0.0;  
float value5 = 1f * (boolean)0;
```

Try out these samples on your computer and see if the result matches your expectation. Here's a hint: Two of these five lines will not compile.



Real World Scenario

IDE Software

While studying for the exam, you should develop code using a text editor and command-line Java compiler. Some of you may have existing experience with Integrated Development Environments (IDEs) such as Eclipse or IntelliJ. An IDE is a software application that facilitates software development for computer programmers.

Although such tools are extremely valuable in developing software, they can interfere with your ability to readily spot problems on the exam. For example, when a line of code does not compile, the IDE will often underline it in red, whereas on the exam, you'll have to find the line that does not compile, if there is one, on your own.

If you do choose to study with an IDE, make sure you understand everything it is doing in the background for you. For the exam, you'll need to know how to manually compile code from the command line, and this experience is rarely learned using an IDE. You'll also need to understand why the code does not compile without relying on the tips and suggestions provided by the IDE.

Identifying Your Weakest Link

The best advice we can give you to do well on the exam is to practice writing sample applications that push the limits of what you already know, as much and as often as possible. For example, if the previous samples with float values were too difficult for you, then you should spend even more time studying numeric promotion and casting expressions.

Prior to taking the OCA exam, you may already be an experienced Java developer, but there is a difference between being able to write Java code and being a certified Java

developer. For example, you might go years without writing a ternary expression or using an abstract class, but that does not mean they are not important features of the Java language. You may also be unaware of some of the more complex features that exist within the Java language. On top of that, there are new features to Java 8, such as lambda expressions and default interface methods, which as of this writing very few professional software developers are using.

The Review Questions in each chapter are designed to help you hone in on those features of the Java language that you may be weak in and that are required knowledge for the exam. For each chapter, you should note which questions you got wrong, understand why you got them wrong, and study those areas even more.

Often, the reason you got a question wrong on the exam is that you did not fully understand the concept. Many topics in Java have subtle rules that you often need to see for yourself to truly understand. For example, you cannot write a class that implements two interfaces that define the same default method unless you override the default method in the class. Writing and attempting to compile your own sample interfaces and classes that reference the default method may illuminate this concept far better than we could ever explain it.

Finally, we find developers who practice writing code while studying for the Java certification tend to write better Java code in their professional career. Anyone can write a Java class that can compile, but just because a class compiles does not mean it is well designed. For example, imagine a class where all class methods and variables were declared `public`, simply because the developer did not understand the other access modifiers, such as `protected` and `private`. Studying for the certification helps you to learn those features that may be applicable in your daily coding experience but that you never knew existed within the Java language.

“Overstudying” Practice Exams

Although we recommend reading this book and writing your own sample applications multiple times, redoing practice exams over and over can have a negative impact in the long run. For example, some individuals study the practice exam questions so much that they end up memorizing them. In this scenario, they can easily become overconfident—they can achieve perfect scores on the practice exams but may fail on the actual exam.

If you get a practice exam question correct, you should move on, and if you get it incorrect you should review the part of the chapter that covers it until you can answer it correctly. Remember that for legal reasons the practice exam questions are not real exam questions, so it is important you learn the material the questions are based on.

On the other hand, we recommend you repeat Review Questions as often as you like to master a chapter. Review Questions are designed to teach you important concepts in the chapter, and you should understand them completely before leaving a section. Furthermore, they help improve your ability to recognize certain types of problems present in many code snippets.

Taking the Test

Studying how to take a test can be just as important as the studying the material itself. For example, you could answer every question correctly, but only make it halfway through the exam, resulting in a failing score! If you're not historically a good test taker, or you've never taken a certification exam before, we recommend you read this section because it contains notes that are relevant to many software certification exams.

Understanding the Question

The majority of questions on the exam will contain code snippets and ask you to answer questions about them. For those containing code snippets, the number one question we recommend you answer before attempting to solve the question is:

Does the code compile?

It sounds simple but many people dive into answering the question without checking whether or not the code actually compiles. If you can determine whether or not a particular set of code compiles, and what line or lines cause it to not compile, answering the question often becomes easy.

Checking the Answers

To determine whether the code will compile, you should briefly review the answer choices to see what options are available. If there are no choices of the form "Code does not compile," then you can be reasonably assured all the lines of the code will compile and you do not need to spend time checking syntax. These questions are often, but not always, among the easiest questions because you can skip determining whether the code compiles and instead focus on what it does.

If the answer choices do include some answers of the form "Does not compile due to line 5," you should immediately focus on those lines and determine whether they compile. For example, take a look at the answer choices for the following question:

18. What is the output of the following code?

- Code Omitted -

- A.** Monday
- B.** Tuesday
- C.** Friday
- D.** The code does not compile due to line 4.
- E.** The code does not compile due to line 6.

The answer choices act as a guide instructing you to focus on line 4 or 6 for compilation errors. If the question indicates only one answer choice is allowed, it also tells you at most only one line of code contains a compilation problem and the other line is correct. Although the reason line 4 or 6 may not compile could be related to other lines of code, the

key is that those other lines do not throw compiler errors themselves. By quickly browsing the list of answers, you can save time by focusing only on those lines of code that are possible candidates for not compiling.

If you are able to identify a line of code that does not compile, you will be able to finish the question a lot quicker. Often, the most difficult questions are the ones where the code does in fact compile, but one of the answer choices is “Does not compile” without indicating any line numbers. In these situations, you will have to spend extra time verifying that each and every line compiles. If they are taking too much time, we recommend marking these for “Review” and coming back to them later.

Determining What the Question Is Asking

A lot of times, a question may appear to be asking one thing but will actually be asking another. For example, the following question may appear to be asking about method overloading and abstract classes:

12. What is the output of the following code?

```
1: abstract class Mammal {  
2:   protected boolean hasFur() { return false; }  
3: }  
4: class Capybara implements Mammal {  
5:   public boolean hasFur() { return true; }  
6:   public static void main(String[] args) {  
7:     System.out.println(new Capybara().hasFur());  
8:   }  
9: }
```

It turns out this question is a lot simpler than it looks. A class cannot implement another class—it can only extend another class—so line 4 will cause the code to fail to compile. If you notice this compiler problem early on, you’ll likely be able to answer this question quickly and easily.

Taking Advantage of Context Clues

Let’s face it—there will be things you’re likely to forget on the day of the exam. Between being nervous about taking the test and being a bit overtired when you read a particular chapter, you’re likely to encounter at least one question where you do not have a high degree of confidence. Luckily, you do not need to score a perfect 100% to pass.

One advanced test-taking skill that can come in handy is to use information from one question to help answer another. For example, we mentioned in an earlier section that you can assume a question’s code block will compile and run if “Does not compile” and “Throw an exception at runtime” are not available in the list of answers. If you have a piece of code that you know compiles and a related piece of code that you’re not so sure about, you can use information from the former question to help solve the latter question.

Use a similar strategy when a question asks which single line will not compile. If you're able to determine the line that does not compile with some degree of confidence, you can use the remaining code that you know does compile as a guide to help answer other questions.

By using context clues of other questions on the exam, you may be able to more easily solve questions that you are unsure about.

Reviewing Common Compiler Issues

The following is a brief list of common things to look for when trying to determine whether code compiles. Bear in mind that this is only a partial list. We recommend you review each chapter for a comprehensive list of reasons that code will not compile. Also, if you have not finished reading the book, you should set aside this list and return to it when you are preparing to take the exam.

Common Tips to Determine if Code Compiles:

- Keep an eye out for all reserved words. [Chapter 1]
- Verify brackets—{ }—and parentheses—()—are being used correctly. [Chapter 1]
- Verify new is used appropriately for creating objects. [Chapter 1]
- Ignore all line indentation especially with if-then statements that do not use brackets { }. [Chapter 2]
- Make sure operators use compatible data types, such as the logical complement operator (!) only applied to boolean values, and arithmetic operators (+, -, ++, --) only applied to numeric values. [Chapter 2]
- For any numeric operators, check for automatic numeric promotion and order of operation when evaluating an expression. [Chapter 2]
- Verify switch statements use acceptable data types. [Chapter 2]
- Remember == is not the same as equals(). [Chapter 3]
- String values are immutable. [Chapter 3]
- Non-void methods must return a value that matches or is a subclass of the return type of the method. [Chapter 4]
- If two classes are involved, make sure access modifiers allow proper access of variables and methods. [Chapter 4]
- Nonstatic methods and variables require an object instance to access. [Chapter 4]
- If a class is missing a default no-argument constructor or the provided constructors do not explicitly call super(), assume the compiler will automatically insert them. [Chapter 5]
- Make sure abstract methods do not define an implementation, and likewise concrete methods always define an implementation. [Chapter 5]

- You implement an interface and extend a class. [Chapter 5]
- A class can be cast to a reference of any superclass it inherits from or interface it implements. [Chapter 5]
- Checked exceptions must be caught; unchecked exceptions may be caught but do not need to be. [Chapter 6]
- try blocks require a catch and/or finally block for the OCA exam. [Chapter 6]

We have listed the chapter each tip is found in so that you can go back and review any that you do not fully understand. Once you've determined that the code does in fact compile, proceed with tracing the application logic and trying to determine what the code actually does.

Applying Process of Elimination

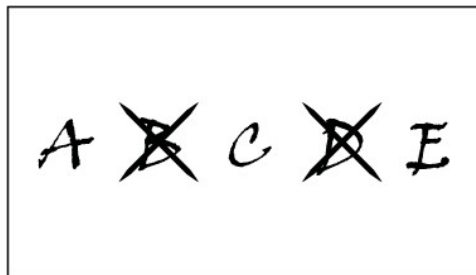
Although you might not immediately know the correct answer to a question, if you can reduce the question from five answers down to three, your odds of guessing the correct answer will be markedly improved. For example, if you can reduce a question from four answers to two answers, you double your chances of guessing the correct answer. In this section, we will discuss how to apply the process of elimination to help improve your score.

Using the Provided Writing Material

Depending on your particular testing center, you may be provided with a stack of blank paper or a whiteboard to use to help you answer questions. If you sit down and are not provided with anything, please make sure to ask for such materials.

After determining whether a question compiles and what it is asking for, you should then jot down a list of all the answers. You should then proceed to cross out the ones you know are not correct. We provided a sample of what this might look like in Figure B.1.

FIGURE B.1 Eliminating answer choices



If you're using paper and you decide to come back to this question, be sure to write down the question number and save it for later. If you're using a whiteboard and decide to come back to a question later, you may have to redo some of the work, given the limited space on a whiteboard. For those questions you want to come back to later, we suggest jotting down the remaining answer choices on the side of the whiteboard. Some test-taking

software allows you to mark and save which answer choices you've eliminated, although in our experience this does not always work reliably in practice.



Although you aren't allowed to bring any written notes with you into the exam, you're allowed to write things down you remember at the start of the exam on the provided writing material. If there's a particular facet of the Java language that you have difficulty remembering, try memorizing it before the exam and write it down as soon as the exam starts. You can then use it as a guide for the rest of the exam. Of course, this strategy only works for a handful of topics, since there's a limit to what you're likely to remember in a short time.

For example, you may have trouble remembering the list of acceptable data types in switch statements. If so, we recommend you memorize that information before the exam and write it down as soon as the exam starts for use in various questions.

Understanding Relationships Between Answers

The exam writers, as well as the writers of this book, are fond of answers that are related to each other. We can apply the process of elimination to remove entire sets of answers from selection, not just a single answer. For example, take a look at the following question:

22. What is the output of the following application?

```
3: int x = 0;
4: while(++x < 5) { x+=1; }
5: String message = x > 5 ? "Greater than" : "Less Than";
6: System.out.println(message+" "+x);
```

A. Greater than,5
B. Greater than,6
C. Greater than,7
D. Less than,5
E. Less than,6
F. Less than,7

In this question, notice that half of the answers output Greater than, whereas the other half output Less than. Based on the code, as well as the answers available, the question cannot output both values. That means if you can determine what the ternary expression on line 5 evaluates to, you can eliminate half the answers!

You might also notice that this particular question does not include any "Does not compile" or "Code throws an exception at runtime" answers, meaning you can be assured this snippet of code does compile and run without issue. If you have a question similar to this, you can compare the syntax and use this as a guide for solving other related questions.

Guessing the Correct Answer

Unlike with some other standardized tests, there's no penalty for answering a question incorrectly versus leaving it blank. If you're nearly out of time, or you just can't decide on an answer, select a random answer and move on. If you've been able to eliminate even one answer, then your guess will be better than blind luck.

Answer All Questions!

You should set a hard stop of 5 minutes of time remaining on the exam to ensure that you've answered each and every question. Remember, if you fail to answer a question you'll definitely get it wrong and lose points, but if you guess, there's at least a chance you'll be correct. There's no harm in guessing!

When in doubt, we generally recommend picking a random answer that includes "Does not compile" if available, although which choice you select is not nearly as important as making sure to not leave any unanswered questions on the exam!

Optimizing Your Time

One of the most difficult test-taking skills to master is balancing your time on the exam. Although Oracle often varies the precise number of questions on the exam and the amount of time you have to answer them, the general rule of thumb is that you have about one and half minutes per question.

Of course, it can be stressful to frequently look at the time remaining while taking the exam, so the key is pacing yourself. Some questions will take you longer than two minutes to solve, but hopefully others will only take less than a minute. The more time you save on the easier questions, the more time you'll have for the harder questions.

Checking the Time Remaining

The exam software includes a clock that tells you the amount of time you have left on the exam. We don't recommend checking the clock after each and every question to determine your pace. After all, doing such a calculation will waste time and probably make you nervous and stressed out. We do recommend you check the time remaining at certain points while taking the exam to determine whether you should try to increase your pace.

For example, if the exam lasts two hours and is 90 questions long, the following would be a good pace to try to keep.

- 120 Minutes Remaining: Start exam.
- 90 Minutes Remaining: One third of the exam finished.

- 60 Minutes Remaining: Two thirds of the exam finished.
- 30 Minutes Remaining: First pass of all questions complete.
- 5 Minutes Remaining: Finished reviewing all questions marked for “Review.” Select answers to all questions left blank.

As you’re taking the exam you may realize you’re falling behind. In this scenario, you need to start allotting less time per question, which may involve more guessing, or you’ll end up with some questions that you never even answered. As discussed in the previous section, guessing an answer to a question is better than not answering the question at all.

Skipping Hard Questions

If you do find you are having difficulty with a particular set of questions, just skip them. The exam provides a feature to mark questions for “Review” that you can later come back to. Remember that all questions on the exam, easy or difficult, are weighted the same. It is a far better use of your time to spend five minutes answering ten easy questions than the same amount of time answering one difficult question.

You might come to a question that looks difficult and immediately realize it is going to take a lot of time. In this case, skip it before even starting on it. You can save the most difficult problems for the end so that you can get all the easy ones solved early on. Of course, you shouldn’t mark every question for “Review,” so use that sparingly. For example, if you only need 30 more seconds to solve a specific question, it is better to finish it so you do not have to come back to it later. The trick is to not get stuck on a difficult question for a long period of time.

Improving Your Test-Taking Speed

Answering certification exam questions quickly does not come naturally to most people. It takes a bit of practice and skill to look at a question, a code sample, and 4–6 answers, and be able to answer it within a minute or two. The best way to practice is to keep solving the review questions at the end of each chapter until you can read, understand, and answer them in under a minute.

Once you’ve completed all of the material and practiced with the review questions enough that you can answer them quickly and correctly, you should try one of the three 60-question practice exams that come with this Study Guide. You should treat it like the real exam, setting aside two hours and finishing it in one sitting.

Although we recommend you try to avoid taking the practice exams so much that you memorize the questions and answers, we do recommend you keep taking them until you can finish each practice exam in under two hours. Remember not to move on to the next one until you can pass the previous exam in the allotted time. If not, study more and go back to drilling on the Review Questions. The idea is that you want to be good at quickly reading through the question, honing in on the key concept the question is asking, and being able to select the answer that best represents it.

Getting a Good Night's Rest

Although a lot of people are inclined to cram as much material as they can in the hours leading up to the exam, most studies have shown that this is a poor test-taking strategy. The best thing we can recommend you do before the exam is to get a good night's rest!

Given the length of the exam and number of questions, the exam can be quite draining, especially if this is your first time taking a certification exam. You might come in expecting to be done 30 minutes early, only to discover you are only a quarter of the way through the exam with half the time remaining. At some point, you may begin to panic, and it is in these moments that these test-taking skills are most important. Just remember to take a deep breath, stay calm, eliminate as many wrong answers as you can, and make sure to answer each and every question. It is for stressful moments like these that being well rested with a good night's sleep will be most beneficial!